

ECE 315

Steven Knudsen, PhD, PEng

Lecture 3 :

- Microprocessor/controller architecture review
- Examples relevant to ECE 315



UNIVERSITY
OF ALBERTA

Last Lecture

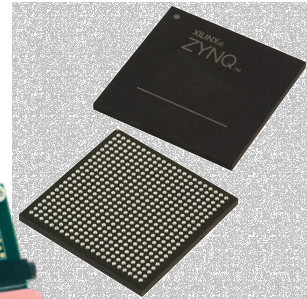
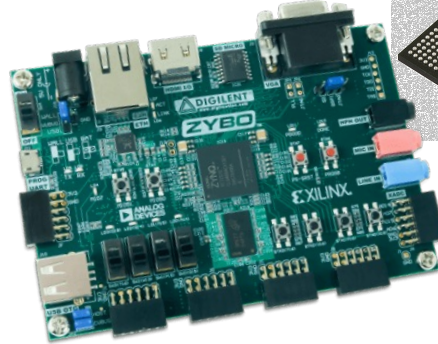
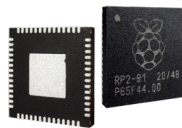
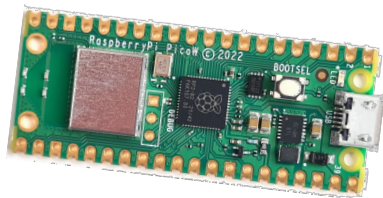
- Embedded Systems overview
 - Importance of embedded systems
 - Examples
- Challenges
 - SWaP
 - Constrained “everything”

This Lecture

- Microprocessor/controller architecture review
- Examples relevant to ECE 315

This Lecture

- Starting on microprocessors, microcontrollers
- Basic architectures
- ARM; Zynq-7000, Pico



Learning Objectives

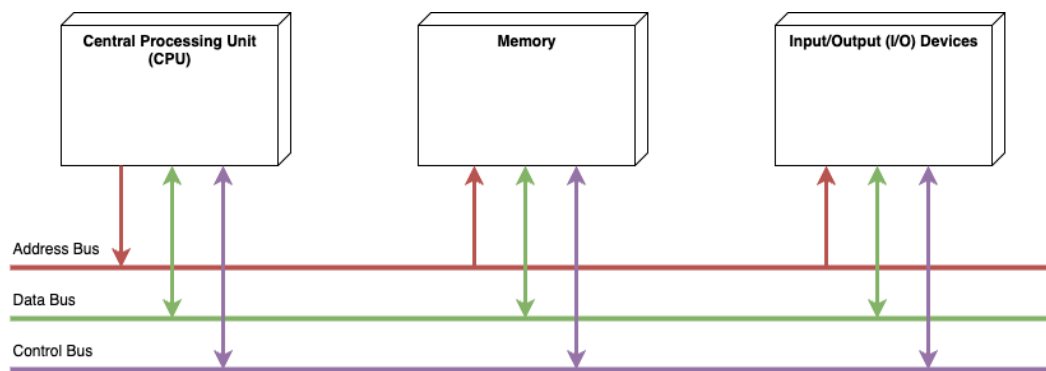
- Identify Harvard, von Neumann, and hybrid architectures
- Know the difference between a microprocessor and a microcontroller
- Understand memory architectures and different types and their purpose
- Understand registers and memory-mapped registers
- Understand how a program is run at the opcode level
 - We will not do assembly programming, but we will think at this level
- Know about and how to work with peripherals that are part of a system-on-a-chip (SoC) and that are external to a micro-processor or -controller

Giant State Machines

From ECE 212 or MCTR 210, we know that microprocessors, microcontrollers and computers are giant state machines

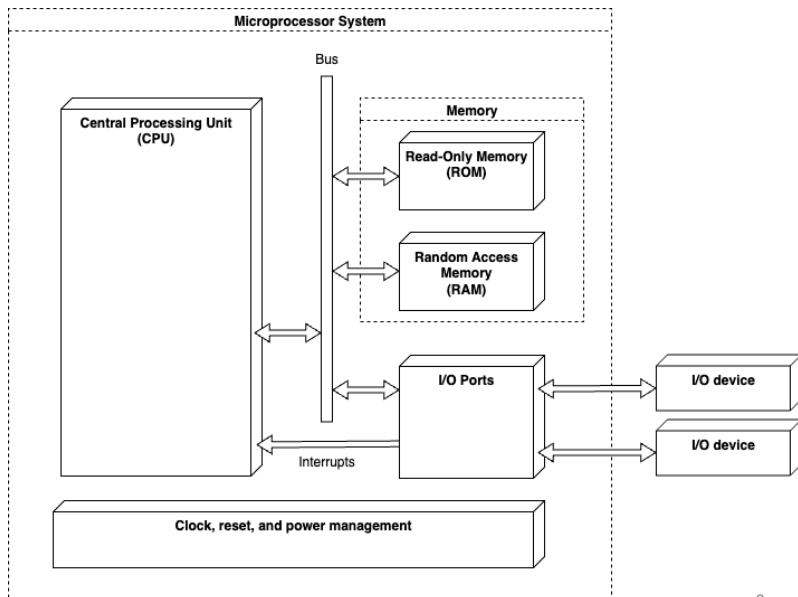
Let's start with a high-level view of what a micro-computer looks like...

Abstract computer



315 Model

This is closer to the model we will consider for computer interfacing, with a focus on embedded devices



Definitions

- A computer is a highly complex synchronous finite state machine.
- **Bus** = a logical grouping of wires that carry related signals; e.g., an 8-bit data bus has a width of 8 bits, and thus moves 1 byte at a time.
 - **Data bus** — moves information between system components, usually one word wide
 - **Address bus** — for CPU to indicate which memory or I/O location is being accessed
- **Control signals** — allow CPU to communicate system state to other components; e.g., reading vs. writing data
- **Clocks** — One or more frequency-stable square-wave clock signals are required to synchronise and initiate events.

Information in a Computer

- **Binary digits** can be used to represent individual values or characters
- **Groups of them** can represent numbers (floating point, signed or unsigned integers, complex numbers, etc.) or character strings (words, sentences)
- More **complex combinations** can represent abstract entities, like documents, spreadsheets, database entries, etc,
- Information is **stored in memory** or transmitted via **buses** or I/O **channels**.

Representing State and Instructions

- The computer, and in particular the CPU and specific logic circuits, maintains state
- Inputs to the state machine are considered to be “instructions” that together comprise a computer “program”
 - You are well-aware of Python and now C programs and programming
 - Languages like them are a high-level abstraction of the state machine inputs that cause changes in state
 - A change in state or a series of changes results in some desired action, such as saving data to memory (RAM) or a signal sent to an I/O device

Representing State and Instructions

- Who defines the inputs and outputs related to state, the instructions
- We, the designers and programmers do
- For example, an instruction to the CPU to write (move) a word to a register might be defined as

MV = 0b1001101100011010

where 0b indicates this is a binary, base-2 word



Adapted from ECE 212 lecture by Dr. J. Sit, 2025

12

This is a made-up instruction

Programming Languages

- High-level programming languages like C, Python, Java are used to provide a high-level abstraction for the low-level machine instructions
- Assembly language is the lowest-level abstraction for machine instructions
 - Provides a one-to-one mapping
- A compiler and linker translate one or more program files into low-level machine instructions and link them with others that are pre-existing and a part of libraries
- The resulting programs are stored in program memory
 - Often this is non-volatile storage like FLASH

Oct 2025	Oct 2024	Change	Programming Language		Ratings	Change
1	1			Python	24.45%	+2.55%
2	4	▲		C	9.29%	+0.91%
3	2	▼		C++	8.84%	-2.77%
4	3	▼		Java	8.35%	-2.15%
5	5			C#	6.94%	+1.32%
6	6			JavaScript	3.41%	-0.13%
7	7			Visual Basic	3.22%	+0.87%
8	8			Go	1.92%	-0.10%
9	10	▲		Delphi/Object Pascal	1.86%	+0.19%
10	11	▲		SQL	1.77%	+0.13%
11	9	▼		Fortran	1.70%	-0.10%



TIOBE Index for October 2025 <https://www.tiobe.com/tiobe-index/>

What are the top programming languages

Top Embedded Programming Languages

This can be very subjective, and people will be passionate, so beware...

geeksforgeeks.org	ChatGPT.com	Google Search AI	Claude.ai
C	C	C	C
C++	C++	C++	C++
Java*	Rust	Python	Assembly
Python	Python	Java*	Python
Rust	Java*	Rust	Rust
Ada	Ada	Assembly	Ada
Lua	Lua	Ada	Java*
B#	Embedded C++	MicroPython	Lua
Verilog	Assembly	Go	Javascript
Embedded C++	Other domain specific	Javascript	Verilog/VHDL



* Embedded Java

15

Specifying Numerical Values in Source Code

- We will focus on the C language as it is less abstracted from the true “machine” code that defines the instructions the computer interprets
- When we want to specify numerical values, we use one of several bases – decimal, binary, octal, or hexadecimal

Example: To specify the numeric value ten, we can write this as:

- `10` – **decimal** by default
- `0x0A` – the `0x` prefix denotes a **hexadecimal** value follows
- `0b1010` – the `0b` prefix denotes a **binary** value follows
- `012` – a leading zero on an integer denotes **octal**

Characters – ASCII

- We will use only ASCII for this course
 - 128 symbols are represented
- Unicode is an extension of ASCII
 - As of version 16, there are 292,531 assigned characters

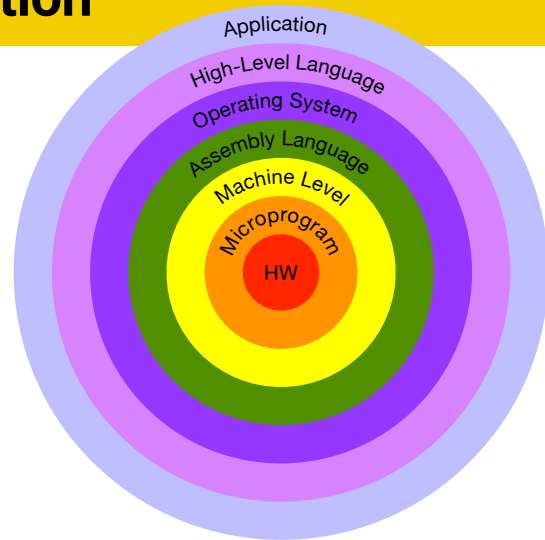
Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	\$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

<https://www.sciencebuddies.org/cdn/references/ascii-table.png>

17

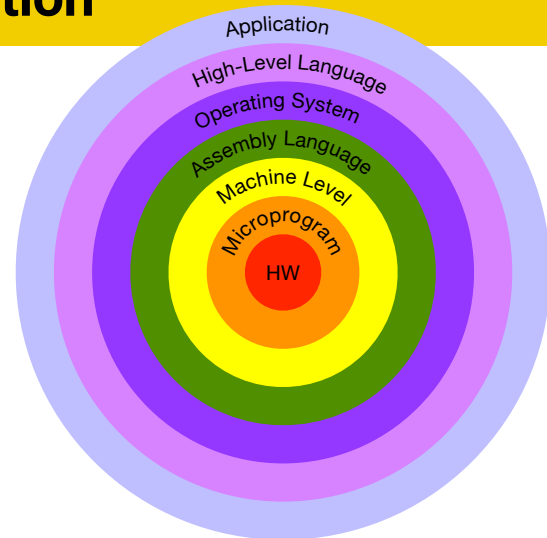
Useful Computer Abstraction

- The outer layers are high-level, closer to the user
- The inner layers are low-level, closer to the circuitry
- We can't alter the hardware (HW), which is made of electronic circuits, gates, flip-flops, etc.
- We can't access the Microprogram level (manufacturer uses that)



Useful Computer Abstraction

- We can program down to the “Machine Level” aka machine code level
- There we write binary-encoded **instructions** that tell the CPU what operation to execute
- The machine code is executed as a series of micro-instructions in the Microprogram level



Classic Computer Architectures

- Historically there are two kinds of computer architecture

Von Neumann

Harvard

von Neumann

- John von Neumann was a Hungarian and American mathematician, physicist, computer scientist and engineer.
- Super smart, made major contributions to multiple fields
- Developed the computer architecture named after him

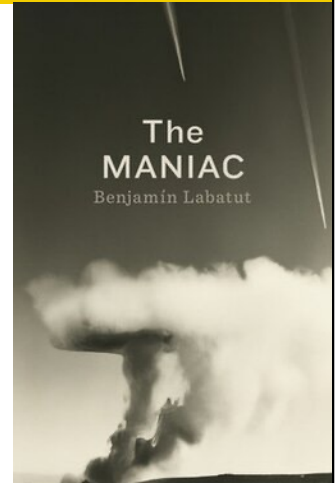


By LANL - <http://www.lanl.gov/history/atomicbomb/images/NeumannL.GIF>
(archive copy at the Wayback Machine), Attribution, <https://commons.wikimedia.org/w/index.php?curid=3429594>

21

The Maniac - book

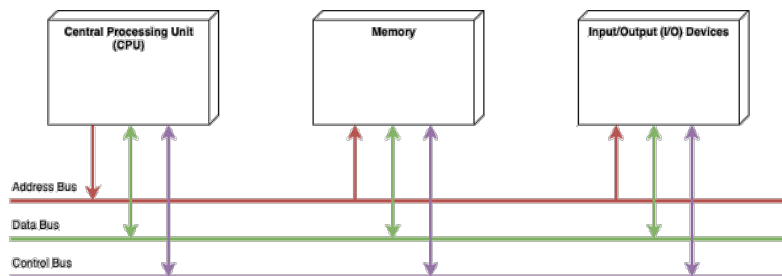
- In case you are interested, there is recent book that is a fictional account of John von Neumann by Benjiman Labatut
- This and his previous book, When We Cease To Understand The World, are crazy good reads
- Not entirely fictional, more of a biography with fictionalized parts...



22

von Neumann Architecture

- Single-bus model
 - Defined as a logical grouping of signal-bearing media (wires usually)
 - Here we group the address, data, and control lines as a single bus
- Only one memory that stores program and data



Adapted from ECE 212 lecture by Dr. J. Sit, 2025

23

We've already seen this.

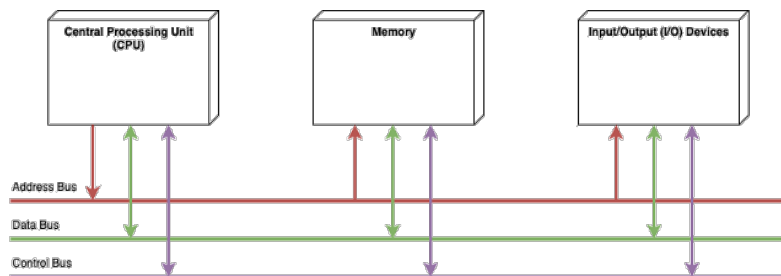
von Neumann Architecture

Pros

- Simple
- Programs can be treated as data
- Simple programming language model and tools (compiler, etc.)
- Cost-effective

Cons

- Single bus is a bottleneck, limits performance
- Programs and data in same memory is a security risk
- Sequential access to program and data; slow

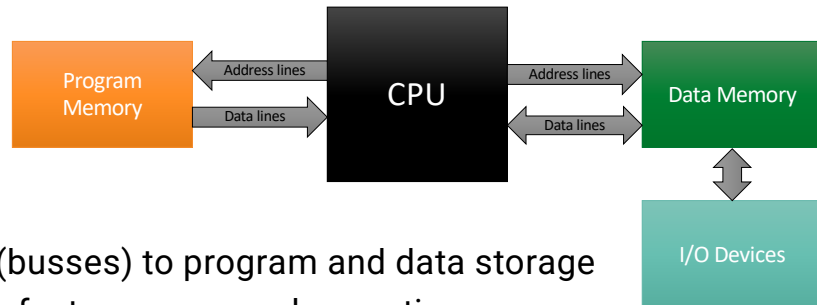


Adapted from ECE 212 lecture by Dr. J. Sit, 2025

24

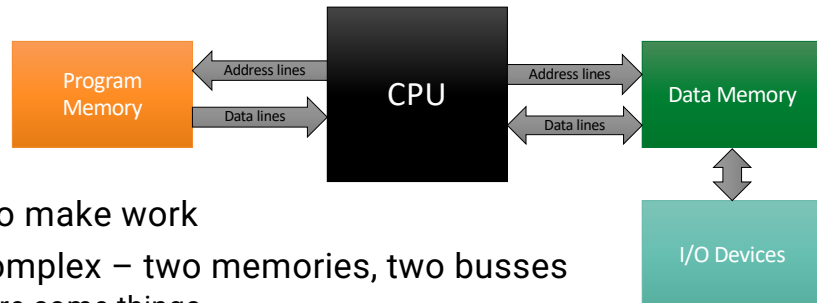
We've already seen this.

Harvard Architecture



- Has separate paths (busses) to program and data storage
- Potentially allows for faster access and execution
 - Instructions and data can be fetched in parallel

Harvard Architecture



- Can be challenging to make work
- Is physically more complex – two memories, two busses
 - May still need to share some things
 - More expensive
- Modern microprocessors are not pure von Neumann or Harvard, but a mix of the two

Advanced RISC Machines – ARM

- A very popular, common computer architecture was developed by companies that eventually became ARM starting in the early 80s
- It's not perfectly correct to say "computer architecture" because ARM licenses a family of Reduced Instruction Set Computer (RISC) instruction set architectures
 - ARM specifies the instructions and companies build hardware to execute them
- Over 200 billion ARM-based computers sold so far (at least!)

Zybo Z-7010

- Main system used for labs and example in ECE 315
- Single-board computer based on Xilinx Zynq Z-7000 SoC

Zybo Features

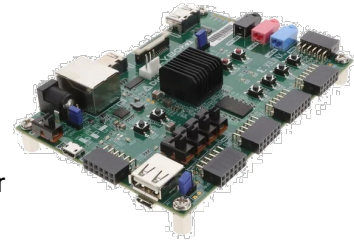


ZYNQ Processor

- 667MHz dual-core Cortex-A9 processor
- DDR3L memory controller with 8 DMA channels and 4 High Performance AXI3 Slave ports
- High-bandwidth peripheral controllers: 1G Ethernet, USB 2.0, SDIO
- Low-bandwidth peripheral controller: SPI, UART, CAN, I2C
- Programmable logic equivalent to Artix-7 FPGA

Memory

- 1 GB DDR3L with 32-bit bus @ 533 MHz (1066 MHz)
- 16 MB Quad-SPI Flash with factory programmed 128-bit random number and 48-bit globally unique EUI-48/64™ compatible identifier
- microSD slot



See <https://digilent.com/>

29

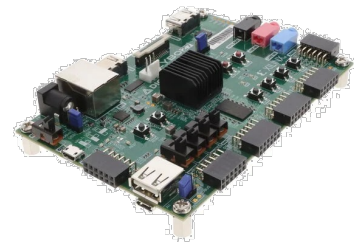
Zybo Features

Power

- Powered from USB or any 5V external power source

USB and Ethernet

- Gigabit Ethernet PHY
- USB-JTAG Programming circuitry
- USB-UART bridge
- USB 2.0 OTG PHY with embedded Host* and Device capabilities



Audio and Video

- HDMI sink port (input) with CEC (Zybo Z7-20) and without CEC (Zybo Z7-10)
- HDMI source port (output) with CEC (Consumer Electronics Control)
- Audio codec with stereo headphone, stereo line-in, and microphone jacks



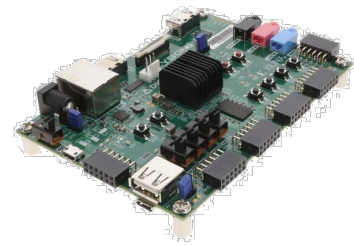
See <https://digilent.com/>

30

Zybo Features

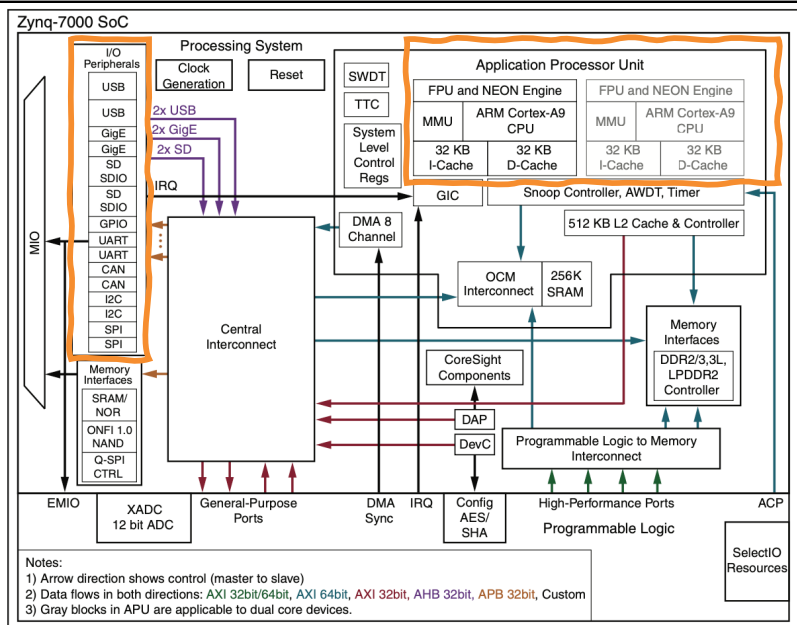
Switches, Push-buttons, and LEDs

- 6 push-buttons
- 4 slide switches
- 5 LEDs
- 2 RGB LEDs
- Expansion Connectors
- 6 Pmod ports on the Zybo Z7-20 (5 on the Zybo Z7-10)



Xilinx Zynq-7000 System-on-a-chip (SoC)

- One or two A9 ARM processors
- Rich set of I/O peripherals
- Multiple memory interfaces
- Multiple communication interfaces
- FPGA fabric



<https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>

Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

UG585_ct_01_060618

32

Cortex-A9

- One of the ARM Cortex-A family of processors
- Supports a Memory Management Unit (MMU), which is needed for modern operating systems

64-bit		32/64-bit		32-bit	
Year	Core	Year	Core	Year	Core
2016	Cortex-A34	2012	Cortex-A53	2005	Cortex-A8
2018	Cortex-A65		Cortex-A57	2007	Cortex-A9
2021	Cortex-A510	2015	Cortex-A35	2009	Cortex-A5
2022	Cortex-A715		Cortex-A72	2010	Cortex-A15
2023	Cortex-A520	2016	Cortex-A73	2011	Cortex-A7
	Cortex-A720	2017	Cortex-A55	2013	Cortex-A12
2024	Cortex-A725		Cortex-A75	2014	Cortex-A17
2025	Cortex-A320	2018	Cortex-A76	2016	Cortex-A32
	Cortex-A530	2019	Cortex-A77		
	Cortex-A730	2020	Cortex-A78		
		2021	Cortex-A710		
		2022	Cortex-A510 (refresh)		



UNIVERSITY OF ALBERTA

Wikipedia contributors. "ARM Cortex-A." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 21 Jul. 2025. Web. 19 Oct. 2025.

Next Lecture

- Finish Microprocessor/controller architecture review
- Operating Systems
- PetaLinux
- RTOS Introduction

Questions

